Microprocessors and Microsystems 87 (2021) 104363

Contents lists available at ScienceDirect

Microprocessors and Microsystems

journal homepage: www.elsevier.com/locate/micpro

A scalable and efficient convolutional neural network accelerator using HLS for a system-on-chip design

Kim Bjerge [a,*], Jonathan Horsted Schougaard [b], Daniel Ejnar Larsen [b]

[a] School of Engineering, Aarhus University, Finlandsgade 22, 8200 Aarhus N, Denmark
[b] Department of Engineering, Aarhus University, Finlandsgade 22, 8200 Aarhus N, Denmark

ARTICLE INFO

Keywords:
System-on-chip
FPGA
High-level synthesis
Convolutional neural network
PYNQ

ABSTRACT

This paper presents a configurable convolutional neural network accelerator (CNNA) for a system-on-chip (SoC). The goal was to accelerate inference in different deep learning networks on an embedded SoC platform. The presented CNNA has a scalable architecture that uses high-level synthesis (HLS) and SystemC for the hardware accelerator. It can accelerate any convolutional neural network (CNN) exported from Keras in Python and supports a combination of convolutional, max-pooling, and fully connected layers. A training method with fixed-point quantised weights is proposed and presented in the paper. The CNNA is template-based, enabling it to scale for different targets of the Xilinx Zynq platform. This approach enables design space exploration, which makes it possible to explore several configurations of the CNNA during C and RTL simulation, fitting it to the desired platform and model. The CNN VGG16 was used to test the solution on a Xilinx Ultra96 board using productivity for Zynq (PYNQ). The result gave a high level of accuracy in training with an autoscaled fixed-point Q2.14 format compared to a similar floating-point model. It was able to perform inference in 2.0 s while having an average power consumption of 2.63 W, which corresponds to a power efficiency of 6.0 GOPS/W.

1. Introduction

In recent years, deep learning with convolutional neural networks (CNNs) has been applied in many different fields, such as image classification [1,2], object detection [3,4] and recognition [5]. In most cases, state-of-the-art CNN models run on a server in the cloud. However, with the increase of Internet of Things (IoT), there is a demand for embedding the deep neural networks into mobile edge computing. This is especially true for computer vision systems, when there is a large amount of collected data and analyses of images must be carried out in realtime.

As CNNs continue to be applied to increasingly complex problems, low throughput, latency and energy efficiency present challenges on embedded devices with central processing units (CPUs) or graphical processing units (GPUs). Due to several attractive features, field programming gate arrays (FPGAs) present promising platforms for hardware (HW) acceleration of CNNs as reported in [6–9]. CNNs that are optimised for fixed-point data or use binary neural networks achieve even better performance [10–13]. In general, FPGAs provide better performance than CPUs and have a better energy efficiency than both CPUs and GPUs.

Historically, the long design time and need for HW experts have limited the use of FPGAs. Here, high-level synthesis tools have enabled automatic compilation from imperative high-level programs to low-level specifications in a hardware description language (HDL) [14]. It is, however, still a challenge to accelerate large-scale CNNs [15] on an FPGA as model parameters typically require far more memory than the on-chip capacity of the FPGAs. Another challenge is to find an optimal configuration for a given HW accelerator design due to the long design time.

The scope of our work is to develop a scalable and flexible architecture that can accelerate the inference of CNNs on a multi-processor system-on-chip design (MPSoC). It presents the design of the HW/SW architecture, i.e. the programmable logic that will reside in the FPGA fabric and the design of the software. The architecture is generic so that it can accept major CNNs, e.g. VGG16 [2], which can be exported from the deep learning framework Keras [16]. It is developed in the PYNQ [17] framework using Python and SystemC [18] to create a generic template-based HW accelerator. To find the optimal design, this study uses a SystemC-based simulation to explore the design space of the optimal configuration parameters of the CNNA. The design model is translated to HDL specification using high-level synthesis (HLS). Our

* Corresponding author.
E-mail address: kbe@ase.au.dk (K. Bjerge).

https://doi.org/10.1016/j.micpro.2021.104363
Received 4 December 2020; Received in revised form 2 September 2021; Accepted 28 September 2021
Available online 2 November 2021

paper discusses the precision, speed and power consumption of the accelerator as well as the fixed-point retraining of the CNN.

1.1. Related work

The current state-of-the-art HW-based CNNAs that inspired the architecture presented in this paper will be discussed in this section.

Microsoft developed an architecture model [19] to accelerate CNNs for a cloud server solution with several FPGA cards. The architecture uses a top-level controller to control the dataflow with a peripheral component interconnect (PCI) memory interface. It has multiple input buffers, one kernel weight buffer, a large array of processing element arrays (PEAs) and a data redistribution block. It uses a direct memory access (DMA) channel to load data from PC memory to the buffers. It uses PEA blocks on the FPGA to perform dot product calculations of the values in the input buffer and the weight buffer. The result of the dot product is saved onto the next input buffer. The CNNA described in this paper uses many of the same elements, such as input, weight buffers, PEAs and DMAs to perform accelerated convolution for each layer of a CNN.

ZynqNet [20] is based on the architecture of the Microsoft model. However, it focuses on making it work for both training and inference. It is built for a SoC design instead of a server solution. The proposed solution seems promising, although it appears to have a few bottlenecks due to a purely C-based HLS implementation of the solution. To rectify this, our work presents a SystemC-based HLS implementation that provides more control of the synthesised RTL-code in terms of parallelism and pipelined HW modules through the use of template classes. ZynqNet uses a circular line buffer (CLB) for input data handling and a memory-mapped master interface to get data from the main memory, i.e. weights and input data are transferred using the memory-mapped interface. The CLB in our architecture is optimised with line and shift buffers that enable efficient pipelining and data alignment.

To accelerate and develop CNNs on reconfigurable HW (FPGAs), a survey of the current state-of-the-art toolflows was published in 2018 by Venieris et al. [21]. The survey compares the performance of: fpgaConvNet [22,23], DNNWEAVER [24], Angel-Eye [25], DeepBurning [26] and Caffeine [27]. The listed toolflows cover mapping of the classic AlexNet and VGG16 to the Xilinx Zynq or UltraScale platforms. Caffe, the deep learning framework by Berkeley AI Research, is the most widely supported front end for these state-of-the-art toolflows. In contrast, our work uses the framework Keras, from which the CNN model is exported with quantised weights. The CNN model description and weights are processed by Python scripts and PYNQ to interface the HW acceleration on a Xilinx platform.

The above-mentioned toolflows can be divided into two main categories of architectures: *streaming architecture* and *single computation engine*. FINN-R, fpgaConvNet and DeepBurning are in the category of *streaming architectures* and are similar to the CNNA architecture presented in this paper. This chained and pipelined architecture can achieve high performance, but the optimal HW design needs to be found and synthesised for each specific CNN. The *single computation engine*, on the other hand, executes CNN layers sequentially, which means that the same HW engine can handle many different CNNs. The engine is controlled from SW, and data must be moved from CPU memory to the on-chip FPGA memory during processing of the CNN layers. The advantage of this approach is that the same HW can be used for several CNN architectures without reconfiguration.

FINN-R [11] is an end-to-end deep-learning framework for the fast exploration of quantised neural networks (QNNs). It is a framework based on the FINN accelerator [28], which is a QNN built for FPGA. The FINN-R consists of a cascade of multiple layer accelerators that are optimised for a pipelined *streaming architecture*. This design reduces the transfer of data between the main memory and the accelerators. The difficult part is to balance the layered accelerators to prevent bottlenecks or resource waste. However, the framework does not solve the problem of different throughput for each layer. FINN-R optimises the generated HW using HLS, allowing for fast exploration of QNN to create the perfect accelerator for a specific FPGA target. The approach presented in this paper also uses HLS to translate the CNN model, but to a *single computation engine*. Contrary to the approach by FINN-R, a *single computation engine* is scalable and can be reused for many similar CNN models.

Tunable parameters must be optimised for the available resources of FPGA devices, which is the case for DNNWEAVER, Angel-Eye and Caffeine. Angel-Eye uses a compiler to translate the input CNN to custom instructions. A similar approach is used by DNNWEAVER, which utilises a macro dataflow instruction set architecture and supports FPGAs from both Xilinx and Altera. Instead of a compiler, our method converts the CNN model description to tunable template parameters defining the HW accelerator. The method then performs simulation to explore an optimal acceleration constrained by the FPGA device and template parameters.

Of the above-mentioned toolflows, only fpgaConvNet supports special layers with irregular dataflow, including the inception, residual and dense blocks that are required for the newest deep neural networks such as ResNet [29], DenseNet [30], InceptionNet and GooglLeNet [31].

Single computation engines build their architectures around a PEA with a buffer for handling input data, which could be a CLB or a row buffer. A streaming design, such as FINN-R, uses the output to feed the next accelerator. Consequently, *streaming architecture* has a large memory to cache layered outputs directly to the next input buffer so that the data are ready for the next CNN layer. However, due to limited internal memory, this approach is not feasible for all FPGAs. In such cases, there is a need for reloading the input data from the main memory. An example of this is the Microsoft model, and the same approach is used in our work. Other architectures use the main memory to cache the data between layers. FINN-R and fpgaConvNet, do this for each block of layers.

2. Proposed work

The CNNA [32] developed in this work has some elements in common with the solutions presented above. It uses the main memory to store data between layers and the *single computation engine* approach. In addition, the architecture is built around a PEA with two buffering systems: one for weights and one for input image data. The latter uses a CLB. The above architectures are very similar, but the major difference lies in the details of the CLB, which enables efficient pipelining and data alignment.

The CNN architecture in our work supports any input size and layer depth, stride, zero padding, and windows size. It makes the accelerator more flexible and enables it to run any CNN model that uses convolution, max-pooling and fully connected layers. Specialised layers with irregular dataflow, such as inception, residual and dense layers, are not supported. It can be used with most CNNs during run-time inference without the need for re-compiling. The accelerator is developed to work with PYNQ [17,33] and uses an application programming interface (API) similar to Keras [16]. In summary, this paper makes the following new contributions compared to related work.

- We present a generic CNN architecture consisting of a *single computation engine* with five core elements (weight buffer, data buffer, PEA, pooling block and output handler) to perform FPGA acceleration of CNN models.
- Stitching is used for convolutional layers that are too large to execute in a single processing pass and to split complex convolutions into sub-convolutions.
- Dynamic autoscaling is used during training to minimise the accuracy difference between the floating point and the quantised fixed-point accelerator.

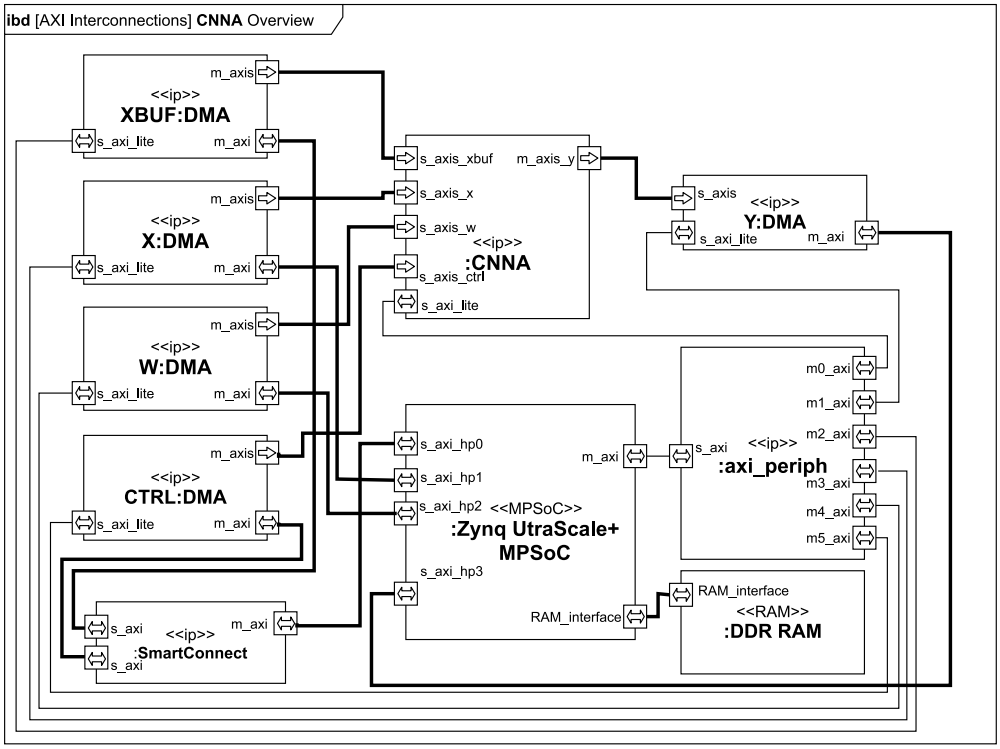

Fig. 1. Block diagram of the system architecture covering CPU (Zynq UltraScale+ MPSoC), memory (DDR RAM), HW IP core accelerator (CNNA), five DMAs for inputs (X), outputs (Y), weights (W), control (CTRL) and splits (XBUF).

- The CNN model description defines the template parameters to create a scalable CNNA architecture constrained by the target FPGA resources.
- A template-based SystemC design with an executable model is proposed for design space exploration. The SystemC model is synthesised to a Xilinx intellectual property (IP) core with HLS and controlled from the host CPU using the PYNQ framework and Python.

3. Design methods

In this section, we will briefly describe the design methods and concepts used as a basis for designing and implementing the architecture of the CNNA.

In our work, SystemC is used with the design flow described in [34, 35, ch. 1]. It is an efficient way in which an IP can be written and verified using HLS. SystemC can model the HW structure and concurrency in a more optimal manner than pure C and C++. It is an IEEE standard (IEEE Std 1666–2011) [18], based on a C++ template library made for HW/SW co-design.

Using template parameters makes the IP core configurable and portable, enabling it to explore different solutions and platforms, whereas custom designs are less flexible. It is much faster to recompile and simulate a template-based IP core than to write a custom IP that may be more optimal. By using SystemC, the desired HW architecture can be controlled and designed via modules with parallel clocked threads. With HLS directives, it is possible to control the synthesised threads and achieve a desired unroll, pipeline, and iteration interval of the synthesised RTL code.

PYNQ [36] is an open-source framework for creating applications on a Zynq MPSoC. The system design in this work is based on PYNQ for controlling the IP directly from Python. This framework is divided into three layers: application, SW and HW.

The application layer, which hosts the user-code, is described in [36, ch. 22]. This is usually Python code in the form of a Jupyter notebook that runs on the ARM CPU inside the Zynq MPSoC. The middle layer in the PYNQ framework is the SW layer, which contains the Python libraries and the interaction with the IP inside the FPGA through the OS drivers. Several drivers are provided through the PYNQ libraries for interacting with the IP. The interface is called an overlay and is used to program the FPGA and manage the IP. The last HW layer in the PYNQ framework is the `bit`-file, which contains the bitstream and is programmed into the FPGA. The interaction between the SW layer and the HW layer is done using DMA or memory-mapped interfaces.

4. System architecture

The SoC design consists of three main elements: the FPGA (i.e. the programming logic (PL)), the dual core CPU and memory (i.e. dynamic random access memory (DRAM)). The goal is to run a CNN consisting of convolutional, max-pooling and fully connected layers computed in the same IP core inside the FPGA logic. The partitioning of CNN computation is performed by the HW, and the SW provides data and control of the HW acceleration for each CNN layer of convolution, max-pooling and fully connected layers. The responsibility of the CPU is to let the user control the HW acceleration so that the IP core is processing CNN layers in correct sequential order. Fig. 1 shows that the system uses DMA to transfer data and weights between the CPU and IP core accelerator. The CPU controls the DMA data block transfer and converts the memory interface to the streaming interface of the IP core.

The system interacts in different manners depending on which scenario it needs to execute. There are three main scenarios: *preprocessing*, *initialisation* and *inference*.

Preprocessing. The first scenario converts the weights to fixed-point and realigns and scales the weights that they are ready for the system to use. Preprocessing also calculates parameters, such as layer output size and layers to be split, which can be done offline on any computer. The weights are transformed from floating-point to fixed-point representation in the chosen format and aligned and rounded off correctly, as described later. Finally, the weights are saved in an h5-file, which is

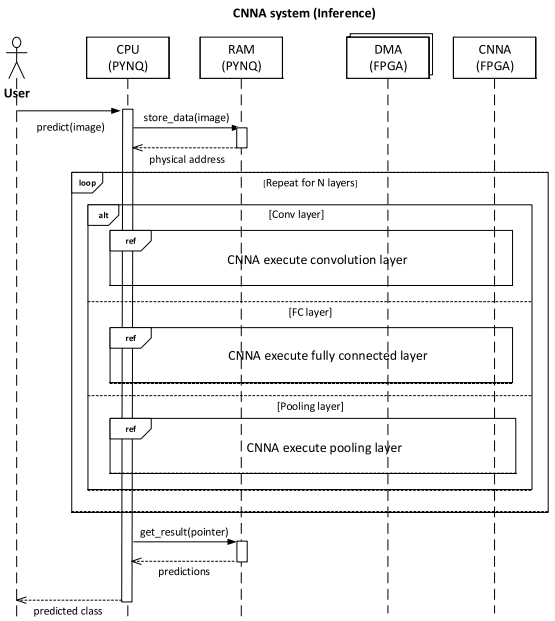

Fig. 2. Sequence diagram of the system of interaction between SW control (PYNQ) and HW accelerator (FPGA) during inference.

the standard format for storing weights in Keras [16]. They can now be transferred to the HW target.

Initialisation. The HW target needs to be configured and initialised for a particular fixed-point resolution by using the synthesised bit-file of the optimised CNNA. The bit-file contains the CNNA IP core and interconnection setup to the CPU for the specified HW target. This is done using a specification of the model in the form of a JSON-file and an h5-file containing weights, which are already realigned and quantised in the preprocessing scenario. It starts by calculating the buffer size and obtaining the properties of the loaded CNNA. When this is done, the SW allocates the needed resources and readies itself for inference by allocating the buffers for each layer in the CNN.

Inference. When using the system, predicting an image will be the most common task, which is shown in the sequence diagram in Fig. 2. Here, the user calls the predict method, which returns the predicted class of the image when the inference is done. The image, which is a parameter to the predict method, is stored internally in a contiguous array, i.e. an array that can be used by the PYNQ DMA library. Depending on the CNN, several layers are executed in the correct order, i.e. convolution, pooling or fully connected layer. The CNN works with different sized pooling and kernel filters in each layer controlled by parameters from the CPU using the CTRL DMA interface. All parameters controlling CNN execution are sent at the start of the predict method.

The convolution is done by the CPU initiating four different tasks in parallel. It sets up the data transfer for the input control data CTRL, the input data X, the output Y and XBUF. Each of the data transfers are handled by the DMA, which streams the content of the buffer from DRAM to the CNNA. The fully connected layer is executed similarly to both pooling and convolution. It starts four different DMAs: one for each of the input data X, the weights W, the output Y and configuration through the CTRL.

Two interfaces are used. The streaming interface used for the DMA is implemented with a functional deterministic guarantee so that no race condition can happen, which makes the entire IP very stable. The AXI streaming interface is used for transmitting the data between the CPU and IP. The other AXI-lite [37] interface, which is a memory-map, is only used for reading status registers.

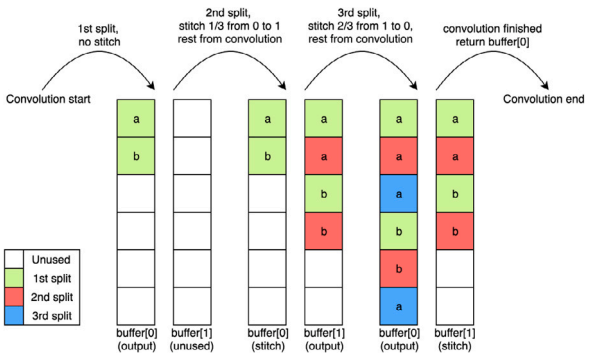

Fig. 3. Example of buffer stitching with a split of three shown for a single pixel.

4.1. Software control and stitching

A CNN has a number of independent kernel filters defining the depth of each convolutional layer. Some convolutional layers are too large to fit in the on-chip memory of the FPGA and must be processed as a single CNNA iteration. This means that they are split into several sub-convolutions using a sub-set of kernel filters in each iteration. However, the result is returned from the IP core with the sub-set depth and, thus, needs to be stitched together with the later result. This is done through the IP core, which has a DMA channel (XBUF) for this purpose, as shown in Fig. 1. An example of stitching can be seen in Fig. 3, which illustrates the output of two pixels, a and b, from a convolution with a depth that needs to be split.

The stitching is done by using two equally sized buffers that both have a size equal to the expected output size. The size in this example is six. The first convolution only uses the first buffer as the output, and pixels a and b are both updated by the DMA. However, only the first third of the depth is calculated in the first convolution. The second convolution calculates the next third of the output. However, these outputs need to be stitched in between the previous outputs. This is done by using the output of the first convolution as a stitch buffer. The IP core is told to use the first part of each pixel and appends the result to each pixel depth-wise. The result of this stitching is sent to the output buffer [1]. The third convolution takes two thirds from the stitch buffer and the last third from the output for each pixel. The output of the stitched convolution is in the buffer, which is the one used as output in the last stitching.

However, the fully connected layers can also be too large to be processed at once, in which case the splits are handled differently. A fully connected layer generates a single value per output. The buffer will be filled with values from the first split when it runs the first time. The second time it runs, it will get the next outputs, which must then be placed after the first split in the buffer. All the splits need to have an adjusted number of bytes to ensure that the correct amount of data is received. When all of the splits have been processed, the result is in the same buffer. This means that the fully connected layer only needs a single buffer, contrary to convolution layers, which need two.

4.2. CNN hardware accelerator

Fig. 4 shows an architecture overview of the main elements of the CNNA. The CNNA works as an accelerator for a single layer at a time. This means that the accelerator needs to be reconfigured for each layer, which is done through the streaming interface, *CTRL*.

The streaming interface *W* is used to load the weights, which can consist of multiple kernels, and caches them in the weight buffer. This means that they can be used multiple times without reloading from DRAM. The streaming interface *X* is used to stream in the input data, which can either be an image or the output from the previous layer.

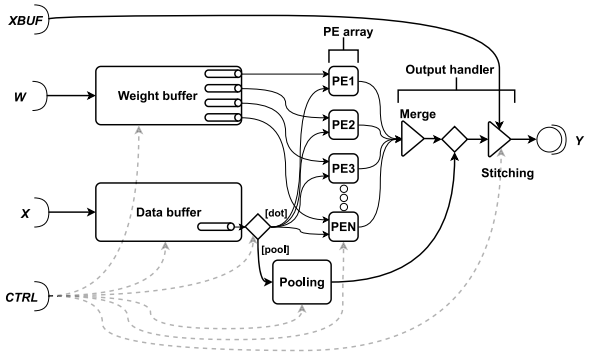

Fig. 4. 1.00 Illustration of the CNNA architecture with five streaming buffer inputs for control (*CTRL*), stitching (*XBUF*), weights (*W*) and data (*X*) as well as one result output buffer (*Y*). A number of PEAs are used to accelerate the pooling and multiplications of each neuron in the network. The CNNA will be executed several times and typically once for each layer during inference.

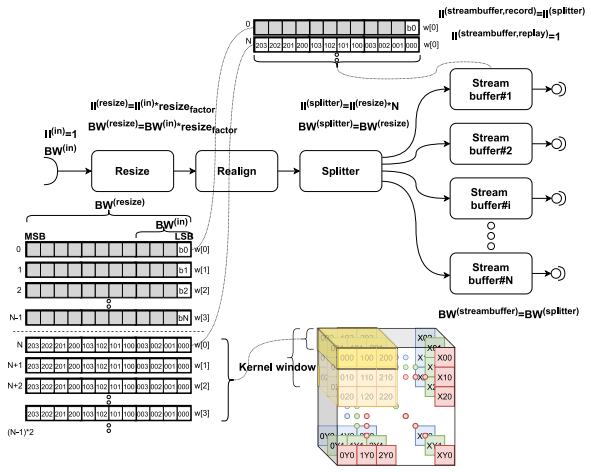

Fig. 5. Weight buffer with alignment illustrates how kernel weight data are aligned so that a specific kernel is placed in the correct spot. The illustration shows how the weight data of kernel 0 are sent to stream buffer 1. The II and BW of the weight input package are changed by a factor of $\text{resize}_{\text{factor}}$. The yellow part of the image cube shows which part of the kernel is sent. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

XBUF is an interface that is used when a convolutional layer is split several times and needs to be stitched together correctly. The last streaming interface is *Y*, which streams out the output values of the operation.

The accelerator is built for three different operations: convolution, pooling and fully connected layers. During convolution acceleration, it uses the weight buffer, data buffer and PEA. The pooling operation is performed using only the data buffer and pooling block. When executing the fully connected layers, the weight buffer and data buffer are simply set to forward the data directly to the PEA, thus, generating a dot product of the two. The following sections describe the five core elements comprising the design of the CNNA.

4.2.1. Weight buffer

The weight buffer is used for caching weight kernels. This caching is necessary because the convolution requires the kernels to be used once for each output pixel.

An illustration of the weight buffer module can be seen in Fig. 5, which shows the modules inside it. The iteration interval (II) and bandwidth (BW) of the weight input package are changed during resizing as illustrated in the figure. It shows how the resize module changes the BW from input BW, $\text{BW}^{(\text{in})}$ by a factor of $\text{resize}_{\text{factor}}$. It also splits the raw package into smaller packages. The realign module also splits the raw package into smaller packages. The splitter separates the data stream into N different stream buffers, each of which has the same BW as the resized BW, $\text{BW}^{(\text{resize})}$. Each stream buffer sends the kernel to a processing element (PE) X times.

The realignment in the weight buffer is complicated. First this is due to the bias value, which uses a complete package. Second the kernels need to match the order in which the three-dimensional window comes from the data buffer, i.e. have the same positions and depths as the data buffer. In Fig. 5, the first package contains the bias values transferred to the weight buffer. It shows that this single value only uses a complete resized package. This is followed by N other bias packages. After all of the bias packages are sent, the weight packages are sent. In this example, the weight packages contain a $3 \times 3 \times 4$ window. The stream buffers receive the values one after the other.

4.2.2. Data buffer

The data buffer is used to handle the input data stream and create the windows on which the convolution or pooling is carried out.

An image typically consists of three channels, RGB, that can be visualised as a three-dimensional cube, which is illustrated in Fig. 6. The image is stored in raster order, i.e. first, pixel (0,0) channel 0, then channel 1 of the same pixel followed by the last channel. This is followed by the same three channels for the pixel one row down, which means that the Z-axis is processed first, then the Y-axis then the X-axis. Raster order is the order in which the image data is streamed to the CNNA.

The CLB can be considered the brain of the CNNA, because it allows it to increase the BW and removes the need to realign the input data for each layer. The parameters that the actor can set through the control interface are:

- *Row size* — the row size of the input image, i.e. the Y-axis length. It is assumed that the image is quadratic.
- *Depth* — the depth is N-channels of the input image, i.e. the Z-axis of the image. This should be dividable by the BW.
- *Stride* — a convolution parameter.
- *Window size* — the size of the window. If the window size is 3, the real window size would be $3 \times 3 \times$depth. This is also a convolution parameter.
- *Zero pad* — a convolution parameter setting the size of the zero padding around the image.
- *Replay* — how many times the CLB should resend a single window.

After setting up the CLB with parameters through the control interface, the image data can flow into it. The CLB consists of two parts: a line buffer for storing N_{lines} previous lines and a shift buffer for storing N_{pixels} previous pixels for each line. These parts are explained in detail below:

Line buffers. The first module in the CLB, where the image data is ordered and stored is the line buffer. This module streams one row of the image with all channels at the same time. The number of line buffers is equal to the maximum window size minus one, $N_{\text{line buffer}} = window_{\text{size}} - 1$. This is because only N previous lines are needed to construct a window. The illustration of the dataflow of the line buffer in Fig. 6 shows that $N-1$ previous lines are stored inside the line buffer and sent out individually. This means that the BW increases with a factor of $window_{\text{size}}$. It also indicates that the buffer is stored circularly. This is handled by the pointer, which can be seen in Fig. 6. This pointer will increase each time a new input is received. After receiving a whole line, the line buffers will rotate, i.e. the first line will be moved to the back the second line will be pushed forward and the pointer will be reset. This is done by multiplexing logic in the implemented design.

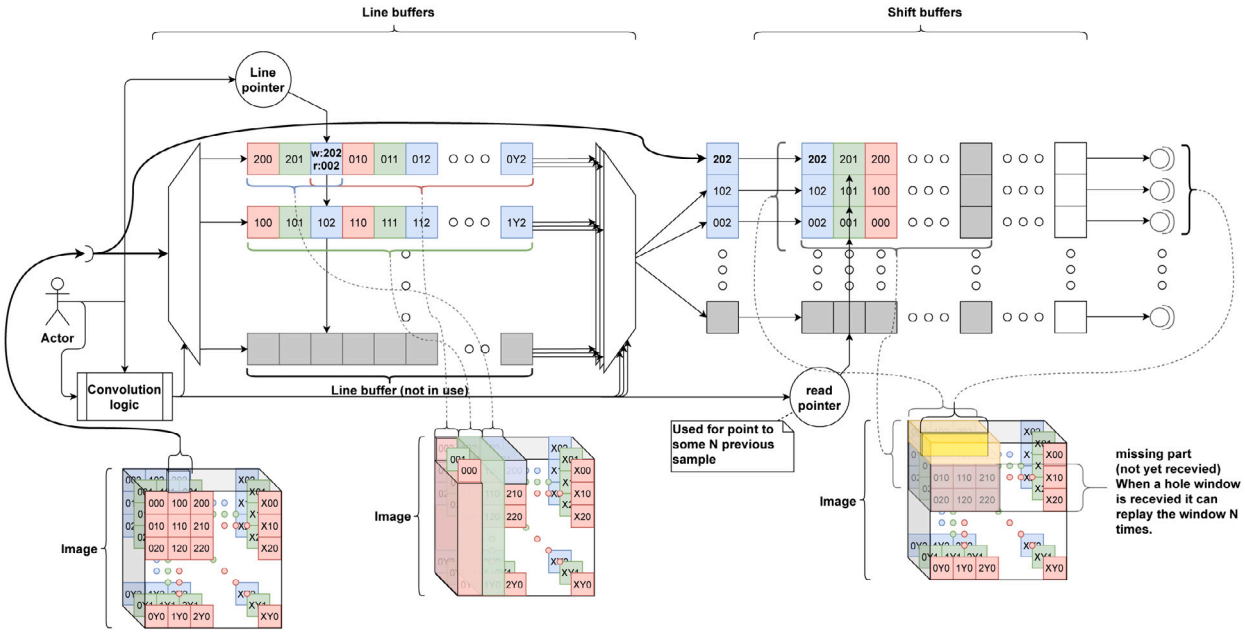

Fig. 6. An illustration of the flow of data through the CLB. It consists of two parts: a line buffer for storing N_{lines} previous lines and a shift buffer for storing N_{pixels} previous pixel for each line. The leftmost image cube illustrates a single pixel 202, which is written for the line buffer. The middle cube illustrates which data are saved in which line buffer and how the new line replaces the first line. The rightmost cube illustrates which data are in the shift buffers. The missing part illustrates the amount of data needed from the line buffers before it has a complete window in the shift buffer. The read pointer on the shift buffers is used for getting the N previous samples and for generating the output from the shift buffers.

Shift buffers. After the line buffers, the data reaches the shift buffers. These buffers are used for getting N previous pixels from each line, i.e. having all the pixels needed for a convolution window, as shown in Fig. 6. The shift buffers have another important function as they replay the window for convolution if there are not enough PEs to run all the dot products in the convolution operation at once. The shift buffers are on-chip RAM-based shift buffers and consist of two pointers. The write pointer is essentially controlled by counting whenever data is written and moving it back to start of the shift buffer when the end has been reached. The read pointer, however, is controlled by logic, which tells the shift buffer that it needs N previous samples. This will be handled by the shift buffer, which also calculates its new positions.

4.2.3. Processing element array

The heart of the CNNA is the PEA. Each PE performs HW acceleration of a dot product with a small range of activation functions, e.g. linear or ReLU [38]. The PE operation can be written as Eq. (1), which is a dot product of the two equal length vectors $\vec{x}$ and $\vec{w}$.

$$PE(\vec{x},\vec{w}) = f\left(\sum_{i=0}^{N-1}(x_i \cdot w_i)\right) \tag{1}$$

Each PE receives data frames in pairs from the weight buffer and data buffer, i.e. one from each. The acceleration of the PE is done by running the multiplications in parallel and totalling the results afterwards, as illustrated in Fig. 7. This data is dotted together and followed by the activation function.

Fig. 7 shows how the PE has two inputs: *W*, the weight input, and *X*, the data input. When the PE has received a frame on both *W* and *X*, the data frames are dotted together and the bias is added to the result. The result is forwarded to the next part, which is the pipelined PE and summation. This part accumulates the result that has been received from the PE dot product. It will keep on accumulating until it receives the last flag. When this happens, it will multiply the accumulated value by a factor set by the actor, i.e. the control interface, and apply the activation, which is also set by the actor through the control interface. Lastly, it is streamed out through port *Y*, and the accumulated result is reset. The whole PE is synthesised with an II of one, which means that new inputs can be processed in each clock cycle. HLS attempts to solve this problem with a summation tree or a cascade of multiply-accumulates (MACs) processed in a long pipeline.

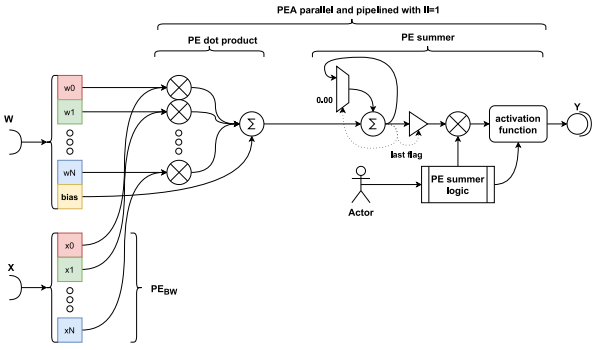

Fig. 7. Illustration of the PEA design, which consists of a parallel multiplier array followed by a summation and an accumulator, scale and the activation function logic. The PE dot product and summation are executed in a parallel and pipeline order.

4.2.4. Pooling

The pooling element is used to accelerate the pooling operation. It gets its input from the data buffer and sends the output to the output handling part, thus, bypassing the PEA, which is not used in pooling.

The reason for placing the pooling operator inside the CNNA is to reuse the CLB hardware. The pooling accelerator receives its input directly from the CLB, and the output from the pooling goes directly to the output handler.

Fig. 8 shows that the pooling block consists of logic for handling the pooling operation, e.g. max-pooling, and RAM for buffering a single pixel. The pooling logic is controlled by the actor and is used for setting the depth of the current image and the size of the window, e.g. $2\times2\times$depth or $3\times3\times$depth. The last parameter controls what type of pooling operator should be run, i.e. max-, min- or average-pooling.

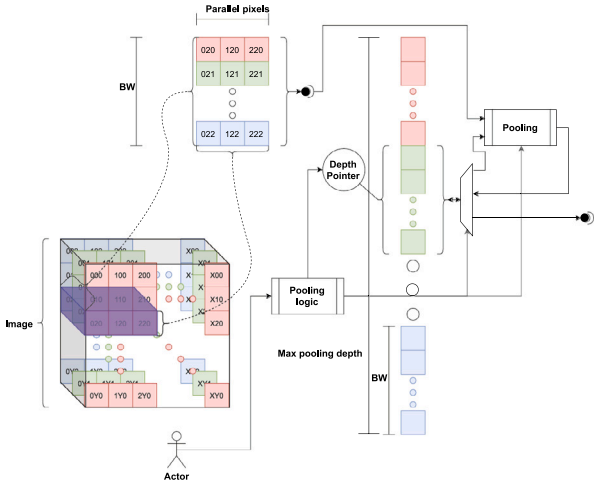

Fig. 8. Illustration of the logic of the pooling accelerator. The data is received as a small slice (the purple cube), which is the output of the CLB. It compares the input data with the current pooling. If it is the first value, it saves it. After pooling, logic is executed. If it is max-pooling it checks if the input is larger than the stored value, and stores the largest one. When a whole window has been run through the accelerator, it will start streaming out the calculated pixels. These steps are repeated for all the windows created by the CLB. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

4.2.5. Output handler

The output handler plays a major role in getting the output of the CNNA into the correct shape and alignment before streaming it out through interface *Y*. Splitting the convolution happens when too many kernels need to be stored in the weight buffer. The output handler merges the results from the PEA with the data to be stitched from the *XBUF*, i.e. if a convolution operation has been split into more than one convolution and needs to have the old output interleaved into the new output. Stitching interleaves data from *XBUF* and the output by aligning the outputs from the PEA, in the right order, as described in Section 4.1. The merge will take data from the PEA which increases the BW with a factor of PEs, followed by a downsize to fit the DMA BW of output port *Y*. The output handler also handles the output of a pooling operation, which simply means forwarding the output of the pooling element.

5. Training for fixed-point

To overcome the challenge of the CNNA using fixed-point weights, an emulation of fixed-point needs to be made for the CNN to be trained and calculated correctly. This is mostly due to the large dynamic range of the weights.

This emulation is shown in Eq. (2), where $Q_{[I.F]}(x)$ is the fixed-point representation of x in the fixed-point format Q[I].[F] [39]. Here, I is the number of integer bits and F is the number of fractional bits. First, the number x is scaled up by 2^F and then rounded off to resolve the limited resolution of fixed-point numbers. This is followed by what is essentially a saturation of the number to the range of the fixed-point number, i.e. between -2^{I+F-1} and $2^{I+F-1}-1$. Lastly, the number is scaled down by the same factor it was scaled up by. This results in a value that can be interpreted correctly by the CNNA.

$$Q_{[I.F]}(x) = \max\Big(-2^{I+F-1},\ \min\big(2^{I+F-1}-1,\ round(x\cdot 2^F)\big)\Big)\cdot 2^{-F} \tag{2}$$

The maximum quantisation error of rounding x (weights) in Eq. (2) is given by:

$$Q_e = \pm 1/2 \cdot 2^{-F} \tag{3}$$

5.1. Quantised weights

The weights are quantised as a constraint to the optimiser, which executes backpropagation [40]. This constraint is set to quantise all of the weights after each update using Eq. (2). This results in the stochastic gradient decent (SGD) update formula shown in Eq. (4), where $Q_{[I.F]}(x)$ is the quantisation function shown in Eq. (2), $W_{ij}^{(l,t=t-1)}$ is the previous weight, $W_{ij}^{(l,t=t)}$ is the new weight, and α is the learning rate.

$$W_{ij}^{(l,t=t)} = Q_{[I.F]}(W_{ij}^{(l,t=t-1)} - \alpha\nabla_{W_{ij}}^{(l,t=t-1)}) \tag{4}$$

However, this introduces a problem that freezes the training. The cause of the problem is that the size of the update to the weights is too small to move from one quantised value to another. The effect of a too-small update change can be seen in the example shown in Eq. (5). It is not possible to update a single weight in Q2.6 with a value smaller than the smallest quantised value, in this case, $2^{-6} = 0.015625$. The example shows a weight with value 1.671875 being updated by a too-small value: 0.0015624. Updating the quantised weight value does not result in a change, which causes the training to freeze.

$$W_{ij}^{(l)} = Q_{[2.6]}(1.671875 - 0.0015624) = 1.671875 \tag{5}$$

To solve this issue, an extra copy of the weights W is saved so that the forward pass, i.e. inference, is calculated using the quantised weights, and the SGD is calculated using unquantised weights. This means that the weights are not stuck between quantisation steps. This is also known as lazy update SGD [41]. In this way, the weights W are saved and the quantised weights WQ are used for the forward pass, which can be seen in Eqs. (6) and (7).

$$W_{ij}^{(l),t=\tau} = (W_{ij}^{(l),t=\tau-1} - \alpha(\nabla_{W}^{(l),t=\tau-1})_{ij}) \tag{6}$$

$$WQ_{ij}^{(l),t=\tau} = Q_{[I.F]}(W_{ij}^{(l),t=\tau}) \tag{7}$$

By using these equations, the optimiser can train the CNN even though the changes are too small to be significant when quantised.

5.2. Dynamic range scaling

The small kernels in the first convolutional layers of the CNN VGG16 have large weights, i.e. close to 1 or −1, but the fully connected layers have very small weights that only use the lowest bits, even in Q2.14. This means that the CNN needs more fractional bits. However, this can be solve by dynamically scaling the weights and the output. This is carried out with integers in [42].

The following will show how this can be carried out on fixed-point values as well. It has been found that the dynamic range of each kernel is almost the same for each layer. This knowledge can be used to add scaling to each layer to change the dynamic range of the weights. For example, based on the given weights

$$W = \begin{bmatrix} 0.11 & 0.024 & -0.30 \\ -0.05 & 0.002 & 0.1 \end{bmatrix}$$

and a fixed-point format Q[I].[F], which, for simplicity, can store a maximum value of 1, denoted $Q_{[I.F]}^{MAX}$, a scaling can be found. To find the scaling needed for a better dynamic range, Eq. (8) can be used. This equation takes the maximum absolute value of the weights and divides it by the maximum value of the fixed-point format.

$$scale^{(l)} = \frac{\max_i\left(|W_i^{(l)}|\right)}{Q_{[I.F]}^{MAX}} = |-0.30| = 0.30 \tag{8}$$

The scaled value of the weights can now be calculated as shown in Eq. (9), which divides the weights by $scale^{(l)}$. This shows that the maximal absolute value is now −1.

$$W_{scale}^{(l)} = \frac{W^{(l)}}{scale^{(l)}} = \begin{bmatrix} 0.367 & 0.08 & -1 \\ -0.167 & 0.00667 & 0.333 \end{bmatrix} \tag{9}$$

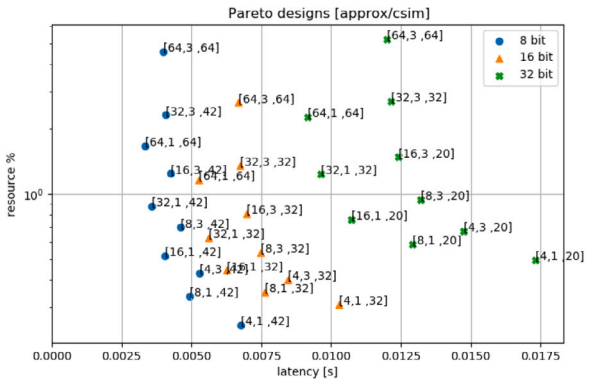

Fig. 9. Results of the C simulation of the combined test of all fixed-point candidates showing the average resource usage of BRAM and DSP versus latency. The $PE_{BW\{\times\}}$ is set to 128 for all solutions. The plotted text for a candidate is in the format $\left[PE_N, DB_{BW\{\times\}}^{(output)}, kernels_N^{(\mathbb{R}^{3\times3\times512+1})}\right]$. The candidates are split into three groups of the word lengths ($data_{size}^{(W)}$) 8-bit, 16-bit and 32-bit versions. It took 2 min to create the estimate for one candidate solution.

Using this scale factor, the output of a layer is calculated as shown in Eq. (10), which has an added multiplication of the quantised value of the scale factor, where $z_{scale}^{(l)}$ is the scaled output of layer l, $W_{scale}^{(l)}$ are the scaled weights, $a^{\vec{l-1}}$ is the output from the previous layer and $scale^{(l)}$ is the scale factor of layer l.

$$z_{scale}^{(l)} = Q_{[I.F]}(W_{scale}^{(l)} \cdot a^{\vec{l-1}}) \cdot Q_{[I_{scale}].[F_{scale}]}(scale^{(l)}) \tag{10}$$

Because of quantisation, it cannot be guaranteed that the outputs are the same, but they should be very similar, i.e. $z^{(l)} \simeq z_{scale}^{(l)}$. The main difference between the scaled and unscaled version is that $z_{scale}^{(l)}$ is better suited for the bit range of the fixed-point format than $z^{(l)}$.

6. Design space exploration

For the purpose of design space exploration, a test bench was written to perform verification and estimation of the FPGA resources needed. The test bench only covered a simulation of the CNNA IP core operations concerning convolution, pooling, and fully connected layers. The stitching algorithm was not verified in the test bench because this would increase the simulation time unnecessarily, and it had no impact on the estimated FPGA resources.

The template-based IP core written in SystemC has a number of parameters that must be selected in order to achieve an optimal solution. The executable model of the IP core gives an approximate estimate of the time performance and FPGA resource usage. When optimising the IP core for an FPGA, it can be an enormous task to generate a design and find the optimal design parameters. It takes approximately one hour to synthesise the HLS code to RTL code. If this were to be carried out for all possible combinations of parameters, it would take weeks, months or even years, because the architecture has such a large number of parameters, e.g. BW between modules, FIFO-depth, the number of PEs, etc. The high-level model of the IP in SystemC can be simulated faster than the RTL code by a factor of 50–200 times, depending on the size of the accelerator. It is possible to use an iterative approach to find the optimal solutions for a certain fixed-point resolution constrained by the given target device by evaluating several simulated solutions.

The design parameters are used for tuning the CNNA design to find a balance between precision, speed and resources. The CNNA tuning parameters used are as follows:

$data_{size}^{(W)}$. The word-length of the fixed-point data format in bits, i.e. I+F. This has an impact on precision.

$PE_{BW\{\times\}}$. The internal BW with an element size of $data_{size}^{(W)}$ used by the CNNA.

PE_N. The number of PEs and the PEA are limited by the size of FPGA fabrics.

$DB_{BW\{\times\}}^{(output)}$. The output BW multiplier after the CLB. Normally, this will be set at an equal value to $CLB_N^{(rows)}$, but can be set to a lower number in order to allow the PE to run with lower BW and potentially have a bigger PEA. The internal BW in the PE will be $DB_{BW\{\times\}}^{(output)} \cdot PE_{BW\{\times\}}$, with an element size of $data_{size}^{(W)}$. The BW used inside the weight buffer is also equal to $DB_{BW\{\times\}}^{(output)} \cdot PE_{BW\{\times\}}$.

$kernels_N^{(\mathbb{R}^{3\times3\times512+1})}$. This is used to calculate

$$WB_{size}^{(buffer)} = \left(\frac{(3\times3\times512)}{DB_{BW\{\times\}}^{(output)} \cdot PE_{BW\{\times\}}} + bias_{size}\right) \cdot kernels_N^{(\mathbb{R}^{3\times3\times512+1})}$$

Here $(3\times3\times512)$ is chosen from the largest layer in the CNN, which, in this case, is the VGG16 [2]. The CNNA IP core needs to be synthesised with the largest kernel size ($K \times K \times depth$) in the CNN layers. The maximum K size kernel filter will be limited by the FPGA on-chip memory.

CNNA tuning can be expressed as a vector, $\vec{\beta}$, with five hyper-parameters, as shown in the equation below:

$$\vec{\beta} = \left[data_{size}^{(W)}, PE_{BW\{\times\}}, PE_N, DB_{BW\{\times\}}^{(output)}, kernels_N^{(\mathbb{R}^{3\times3\times512+1})} \right]$$

To measure the performance of the different CNNA configurations, a simulation was made consisting of five different elements: two pooling operations, two convolution operations and a single fully connected operation. They were executed individually but evaluated together. The five elements were selected to simulate RTL and verify the basic operations in a CNN within a reasonable number of hours.

When looking at the latency for the combined simulation test, i.e. the five simulations carried out consecutively, the dominant candidates all have $DB_{BW\{\times\}}^{(output)} = 1$ regardless of word length (see Fig. 9). The figure shows that the faster the accelerator, the higher the number of PEs.

Two models were created for each configuration, one of which was done using C simulation, i.e. a simulation that used the SystemC HLS code directly. The other was an RTL simulation, which used the RTL code generated from the SystemC model for the most optimal solutions. The latter was clock cycle accurate, and the execution time was precise.

Several candidates were identified and are shown in greater detail in Table 1. The table shows the number of digital signal processing slices (DSPs) and BRAM used, as well as the total latency for C and RTL simulation. The candidates marked with a "-" used more resources than were available on the tested target platform.

The candidates with the lowest latency were synthesised and tested using RTL simulation, which simulates the real HDL code generated. This also gave more precise resource usage, which only differed slightly from those estimated using C simulation. The execution time is also shown and is slightly longer (approx. 2 ms) than the estimated value. On average, the compilation and C simulation time took two minutes for each solution. The HLS synthesisation and RTL simulation took 1 – 7 hours.

The optimal parameters were found using two different fixed-point formats ($data_{size}^{(W)}$): Q2.14 and Q2.6, i.e. a word length of 16 bits and 8 bits, respectively. These were chosen because of the area constraints of the FPGA on the Xilinx Ultra96 board [43]. However, 32-bits would have been possible with a larger FPGA.

Finally, three different configurations of the CNNA were chosen for the final test of the system, one of which used 16-bit fixed-point format Q2.14, while the other two used 8-bit fixed-point format Q2.6.

Table 1
Design space exploration of resource usage and latency of possible CNNA candidates using C and RTL simulation. A "÷" means RTL simulation was performed, but there was insufficient space on the target platform (Ultra96). A "–" means RTL simulation was not performed.

Parameters $\vec{\beta}$	Resource average %	DSPs	DSPs (RTL)	BRAMs	BRAMs (RTL)	Latency [ms]	Latency [ms] (RTL)
[8, 128, 8, 3, 42]	70	384	359	144	249	4.60	6.66
[8, 128, 8, 1, 42]	34	128	125	137	185	4.95	7.28
[8, 128, 16, 3, 42]	124	768	–	152	–	4.26	–
[8, 128, 16, 1, 42]	52	256	45	139	193	4.03	6.66
[16, 128, 8, 3, 32]	54	192	360	233	377	7.47	8.40
[16, 128, 8, 1, 32]	35	64	–	227	–	7.61	–
[16, 128, 16, 3, 32]	81	384	÷	239	÷	6.98	÷
[16, 128, 16, 1, 32]	44	128	293	229	349	6.27	8.52
[32, 128, 8, 3, 20]	94	384	–	355	–	13.19	–
[32, 128, 8, 1, 20]	58	128	165	351	336	12.92	14.53
[32, 128, 16, 3, 20]	148	768	–	359	–	12.42	–
[32, 128, 16, 1, 20]	76	256	325	353	408	10.73	12.81

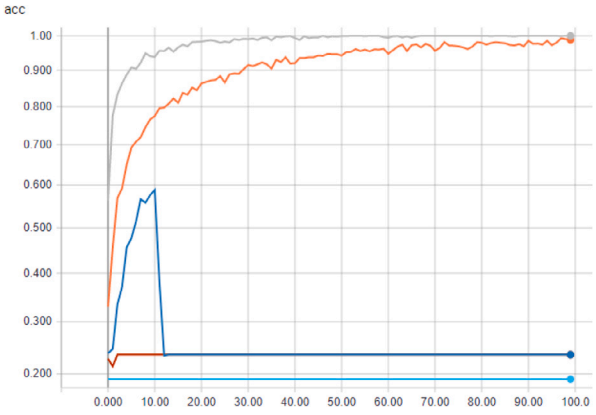

Fig. 10. Training with five classes: grey: floating-point; orange: fixed-point Q2.14 with autoscaling; blue: fixed-point Q2.14 without autoscaling; red at the bottom: fixed-point Q2.6 with and without autoscaling. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

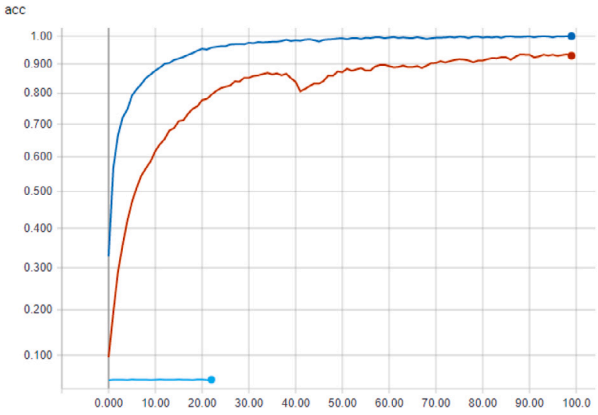

Fig. 11. Training with 29 classes: blue: floating-point; red: fixed-point Q2.14 with autoscaling; light blue: fixed-point Q2.14 without autoscaling. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

7. Results and discussion

The dataset DETECT [44] was used to verify the system. This dataset consisted of 29 classes of micro-invertebrates suspended in alcohol. Only the first five classes were used in the first test, while the second test used all 29 classes. Cifar-100 [45] and ImageNet [46] were used for comparison with other common datasets and to validate the results. The training was carried out over the span of 100 epochs.

The CNN used was VGG16 [2]. The CNN convolutional blocks of VGG16 were followed by two dense fully connected layers with either 4096 or 1024 neurons. Its final fully connected layer had either five or 29 neurons, depending on the number of classes. The training was performed on two fixed-point formats, Q2.14 and Q2.6, and tested on three configurations, which will be denoted CNNA_{16}, CNNA_8^1 and CNNA_8^2. CNNA_{16} used the tuning parameters $\vec{\beta} = [16, 128, 8, 3, 32]$, CNNA_8^1 used $\vec{\beta} = [8, 128, 16, 1, 42]$ and CNNA_8^2 used $\vec{\beta} = [8, 128, 8, 3, 42]$.

The accuracy, performance, and power consumption of the proposed system will be presented and discussed in this section.

7.1. Accuracy

The CNN was trained using a small dataset in order to find suitable candidates faster because it is faster to train with five classes than 29 classes. If the accuracy of a fixed-point format is poor on five classes, it will most likely be poor, or worse, when training on 29 classes. Therefore, initial training was carried out on a small dataset.

Fig. 10 shows that most of the trained models faced issues and obtained low accuracy when using fixed-point format. The only quantised version that obtained a high level of accuracy was the one using the fixed-point format Q2.14. It is unknown why training with fixed-point format Q2.14 and no autoscaling made a sudden dive after 10 epochs. However, it could be caused by the learning rate being too high or too low, or too few neurons in the fully connected layers. The best results were achieved with fixed-point format Q2.14 and autoscaling, which converged towards an accuracy of almost 100%. Not all fixed-point Q2.6 versions could be trained or achieve any useful results.

Table 2
VGG16 training results on five classes.

Type	Autoscale	$N_{neurons}$	Validate	Train
Float	*n/a*	1024	97.5	100.0
Q2.6	Yes	1024	20.3	20.9
Q2.14	No	1024	24.3	23.6
Q2.14	Yes	1024	94.2	98.8
Float	*n/a*	4096	97.9	99.5
Q2.14	No	4096	83.2	83.6
Q2.14	Yes	4096	91.7	97.6

Table 2 shows the results of the training with five classes. Only the training that used Q2.14 with no autoscaling performed well with 4,096 neurons and reached approximately 83%. The table shows the number of neurons in the fully connected layers, N_{neurons}, as well as training and validation accuracy. Validation was performed on a dataset that was not used for training.

A final test was performed on all 29 classes of DETECT with the candidates that performed well in the previous test. The best candidates were the floating-point version for reference and the versions that used fixed-point format Q2.14, both with and without autoscaling. It is evident from Fig. 11 that only the training using fixed-point format Q2.14 and autoscaling achieved promising results. It shows that it

Table 3
Results for the training of the 16-bit fixed-point VGG16 on 29 classes from the datasets DETECT (DET), ImageNet (Image) and Cifar-100 (Cifar).

Type	Data	Auto	$N_{neurons}$	Val.	Train
Float	DET	*n/a*	1024	88.0	100.0
*Q*2.14	DET	No	1024	5.0	5.0
*Q*2.14	DET	Yes	1024	86.4	94.4
Float	DET	*n/a*	4096	84.0	99.4
*Q*2.14	DET	No	4096	5.0	5.1
*Q*2.14	DET	Yes	4096	86.5	92.9
Float	Image	*n/a*	4096	83.0	99.2
*Q*2.14	Image	Yes	4096	79.5	88.1
Float	Cifar	*n/a*	4096	80.5	99.5
*Q*2.14	Cifar	Yes	4096	77.3	89.3

Table 4
Average inference time and variance using VGG16 for five classes using four different IP cores.

	$CNNA_{16}$ 100 MHz	$CNNA_{16}$ 172 MHz	$CNNA_8^1$ 172 MHz	$CNNA_8^2$ 172 MHz
Avg [s]	2.20	1.96	1.22	1.49
Var [$\cdot 10^{-3}$]	0.25	0.30	0.20	0.11

is much more difficult to train the CNNA when using quantisation, because details are lost due to the limited range of the fixed-point numbers. However, it takes many more iterations for the training to reach the same level of accuracy as the floating-point format.

The first 29 classes from ImageNet and Cifar-100 were also used for training. The validation results in Table 3 shows that when comparing the *Q*2.14 format with floating-point the accuracy drops by 3.5% and 3.2%. For DETECT the drop is 4%, which is higher in comparison with ImageNet and Cifar-100 training.

7.2. Performance

The Xilinx Ultra96 board [43] was used to evaluate the performance of the system using a HW clock of 100 MHz and 172.22 MHz for the CNNA IP core. The inference time was measured for different configurations of $CNNA_{16}$, $CNNA_8^1$ and $CNNA_8^2$. The inference times are shown in Table 4. Timing performance was measured on the Ultra96 board during inference of the quantised and trained VGG16 model with five classes. The mean time and variance was an average of 30 measurements. The fastest model, $CNNA_8^1$, took 1.22 s per image, while the slowest, $CNNA_{16}$ at 100 MHz, took 2.20 s per image.

The different layers had different execution times, as shown in Table 5. As expected, the execution time in convolutional layers depended on the number of bits in the fixed-point format. However, pooling took approximately the same time for all tested IPs as pooling is independent of the fixed-point format. The table shows that IP $CNNA_8^1$ obtained the best performance due to the larger number of PEs (16). Note that $CNNA_8^2$ was slightly faster than $CNNA_8^1$ in the convolutional layers, even with fewer PEs, due to the higher BW of the output multiplier. There is a large number of splits (512) in the dense_1 and dense_2 layers, and they consumed more than half of the total execution time for all three CNNA configurations. On average, 32% of the time was used to setup the DMAs from PYNQ, which could be optimised with a scatter-gather DMA. In such a solution, the DMA would initiate transfer for the next location of DRAM data without involving the CPU. A larger FPGA with more on-chip memory could also be a solution to lower the number of splits and optimise the performance further.

7.3. Power consumption

The power consumption of the design with $CNNA_{16}$, $CNNA_8^1$, and $CNNA_8^2$ was measured on the Ultra96 board during inference of the trained VGG16 model with five classes. The measured voltage of the power supply to the board was multiplied with the measured current to compute the power consumption. The mean and maximum power during inference was calculated as a mean of 10 inferences. The power consumption of the IP core is defined as the difference between the Ultra96 board idling and power during inference. The idle power consumption was measured at $P_{\text{idle}} = 3.055$ W over a five-minute period.

Table 5
Time of execution of each VGG16 layer in [ms] using four different IP cores.

Layer	$CNNA_{16}$ 100 MHz	$CNNA_{16}$ 172 MHz	$CNNA_8^1$ 172 MHz	$CNNA_8^2$ 172 MHz
l1_conv1	19.3	17.0	19.9	21.4
l1_conv2	111	84.1	61.3	60.1
l1_pool	18.1	13.7	12.9	17.1
l2_conv1	55.4	42.9	31.3	30.5
l2_conv2	108	81.3	60.2	56.3
l2_pool	8.98	6.87	6.36	8.40
l3_conv1	56.0	43.5	33.1	29.9
l3_conv2	112	84.2	64.2	59.1
l3_conv3	110	85.5	63.0	57.8
l3_pool	4.51	3.48	3.25	4.23
l4_conv1	64.6	51.5	37.9	35.3
l4_conv2	126	97.9	76.0	70.5
l4_conv3	123	102.0	73.5	67.8
l4_pool	2.32	1.83	1.71	2.19
l5_conv1	46.6	41.0	30.7	29.3
l5_conv2	49.1	39.7	29.7	28.1
l5_conv3	45.8	39.5	29.6	27.8
l5_pool	0.74	0.62	0.59	0.69
dense_1	767	737	364	509
dense_2	393	397	197	362
dense_3	1.55	1.62	1.18	1.50

Table 6
Average and peak power consumption in watt of the Ultra96 board and IP core during inference.

	$CNNA_{16}$ 100 MHz	$CNNA_{16}$ 172 MHz	$CNNA_8^1$ 172 MHz	$CNNA_8^2$ 172 MHz
P_{avg}	5.28	5.68	4.71	4.80
P_{peak}	6.60	7.14	5.76	6.35
$P_{IP_{\text{avg}}}$	2.23	2.63	1.66	1.74
$P_{IP_{\text{peak}}}$	3.55	4.09	2.71	3.30

Table 6 shows that the mean power consumption of the Ultra96 board for all tests was between 4.7–5.7 W, out of which the IP core only consumed approximately 2 W. This means that running the IP did not affect average power consumption. However, because they run for a shorter amount of time, the fixed-point IPs with a low number of bits used less energy per inference. The $CNNA_{16}$ with a 100 MHz clock was 0.24 s slower but consumed less power than the version with a 172 MHz clock. Table 6 shows that peak power consumption was almost the same for all tested IPs in the range from 2.7 W to 4.1 W.

Fig. 12 shows that the power consumption was largest at the beginning of the inference, i.e. in the convolution blocks of the CNN. The power consumption dropped during execution of the fully connected layers. This indicates that most of the FPGA logic was in action during convolution, while less logic was used while computing the fully connected layers and pooling. Pooling activity corresponds to the big dips in power consumption in the first half of the inference.

8. Comparison with state-of-the-art CNNs

We have chosen to evaluate our work with the current state-of-the-art toolflows presented in [21], which use a fixed-point resolution of 8 or 16-bits to perform FPGA acceleration of the VGG16 network by targeting the Xilinx Zynq and UltraScale platforms. The purpose of this is to compare our work with other tools that have mapped the same CNN on similar FPGA devices from the same vendor.

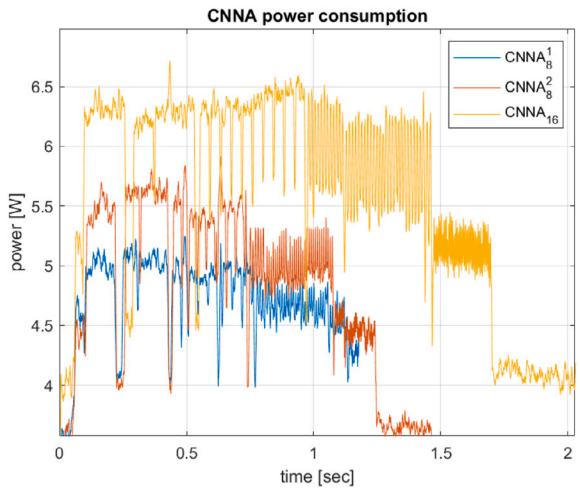

Fig. 12. Power consumption of the tested solution with format CNNA$_8^1$, CNNA$_8^2$, and CNNA$_{16}$ during inference at 172 MHz.

Accuracy. Our fixed-point training method only performed well for 16-bit quantisation. DoReFa-Net [47] proposes a method for training CNNs with low-bit quantisation that demonstrates high accuracy using AlexNet with only 1-bit weights. FINN-R [11] uses quantised neural networks for low-bit quantisation of different CNN topologies with high accuracy. Angel-Eye [25] also proposes a dynamic quantisation strategy, where the network is initially trained with a floating-point format. The radix position of the fixed-point data is chosen differently for each layer based on statistics, and an optimal radix point is chosen. The network is converted back to floating-point and fine-tuned. This method achieves a high level of accuracy for both 16- and 8-bit formats with VGG16.

Performance. To compare the different solutions, the performance needs to be expressed in giga operations per second (GOPS). The performance result is normalised relative to the number of lookup tables (LUTs) and DSPs as a measure of available resources on the target device. This performance density measure is used to compare the VGG16 mapped to different FPGA devices. The throughput performance is calculated as the number of giga operations (GOP) performed by the CNN relative to the inference time in seconds. In the case of the VGG16 network, the total number is 30.76 GOP out of which 30.7 GOP, is performed in the convolutional layers. We have presented the performance of convolutional layers and all layers of the VGG16 model as some solutions do not accelerate the fully connected (FC) layers.

The results are shown in Table 7, which indicates that the CNNA performance (total) is lower than the comparable state-of-the-art architectures. The best performance of our 16-bit solution is 29.1 GOPS. This is lower than the 31.4 GOPS for DNNWEAVER, which has the worst performance of the state-of-the-art solutions. Our lower performance is partly due to the overhead used to setup and split each layer for the DMAs from PYNQ as discussed in Section 7.2. The Ultra96 target used in our evaluation is small and low-cost compared to the ones used in some of the examples, e.g. Zynq XC7Z2045 and UltraScale KU060. If a larger and more expensive target, such as the Xilinx ZCU104 evaluation kit [48] was used, it would be possible to increase the number of PEs, thereby, achieving a higher throughput and performance.

The performance density measure is also lower than most of the other architectures and only similar to DNNWEAVER. Angel-Eye and Caffeine both have a much higher density performance compared to the use of LUT and DSP resources on the FPGA.

The reason for the poor performance concerns three major bottlenecks related to data transfer in the CNNA design. First, the four DMA streams cannot read and write from the DRAM simultaneously, which reduces the DMA burst with $\frac{1}{4}$ of the maximum speed. Second, most state-of-the-art solutions use a large batch size, which increases the performance because weights can be reused for multiple images, which saves reloading the weights for every image. Third, a scatter-gather DMA that automatically loads the next data and weights to the CNNA without involving the software part would increase communication performance. Especially for the FC layers with many splits. The software part could also perform FC layer operations in parallel with the HW to boost performance even more.

Power efficiency. Power efficiency is dependent on both the efficiency of data communication and computation.

The SmartShuttle [49] solution optimises CNN off-chip memory access. Observing that over 80% of energy is consumed by DRAM access, they make a benchmark of the data volume of DMA requests during inference of the 13 convolutional layers in VGG16. Our CNNA16 measures a data volume of 211.7 MB transferred for the same feature layers, including pooling. However, we use more on-chip memory for weight and data buffers than SmartShuttle. As a benchmark, SmartShuttle measures 221.3 MB. When simulated with an on-chip buffer of 512 KB, however, they can lower the DRAM access volume to 160 MB. The design of the CLB in our CNNA ensures that weights are only transferred once from DRAM, which is similar to what SmartShuttle achieves with the weight-reuse-oriented scheme (WRO) they propose. The last three FC layers of the CNNA16 transfers a volume of 273.8 MB and encompasses most of the data communication. This volume of data transfer in the FC layers is not considered by SmartShuttle.

The computation power efficiency is calculated as the number of operations per second, relative to the mean power consumption of the CNNA, which we measured earlier (GOPS/W). Compared to many of the current state-of-the-art accelerators, the CNN accelerator in this work performs quite well in terms of power efficiency. When using 16-bit fixed-point weights at 100 MHz, its total power efficiency is 0.44x lower than Angel-Eye and 0.59x lower than Caffeine. With nearly the same efficiency of 12 GOPS/W, the power efficiency of the convolutional layers is considered comparable with Caffeine. The performance bottleneck in our CNN accelerator is the fully connected layers, where splits are performed 512 times with a high DRAM access. The fpgaConvNet on the Zynq XC7Z020 is least efficiency at 7.3 GOPS/W compared to the CNNA16 with 11.9 GOPS/W. While Angel-Eye's fixed-point 8-bit with 24.1 GOPS/W is the best of all the compared state-of-the-art solutions in terms of efficiency, the 8-bit CNNA with 23.0 GOPS/W is a close second.

Even though the CNNA does not outperform the state-of-the-art solutions, it presents a new *single computation engine* with HLS and template parameters that can be scaled to fit a hardware of a constrained sized. When a CNN architecture is too large to execute in hardware, complex convolution layers are split into sub-convolutions, which introduces some communication overhead as described in Section 7.2. The CNNA model is synthesised to a Xilinx IP core and controlled from the host CPU using the PYNQ framework and Python, which none of the other compared state-of-the-art solutions have covered.

9. Conclusion

In this paper, an architecture for a SoC design was presented that implements the different operations necessary for a deep neural network to perform close to real-time inference. The architecture was implemented using Python and HLS for the IP core and was able to run on the Ultra96 board using PYNQ. The interface for the system is similar to Keras and should be familiar to most engineers working in the field of machine learning.

The CNN can accelerate deep learning algorithms that use any sequence of convolutional, max-pooling and fully connected layers. The layer operations can support many different parameters and could perform inferences using most modern CNNs. The network weights can

Table 7
Table comparing the CNNA with state-of-the-art CNN accelerators: DNNWEAVER [24], fpgaConvNet [22], Angel-Eye [25] and Caffeine [27]. All solutions targets Xilinx Zynq devices except for Caffeine, which uses the Kintex UltraScale FPGA. Power efficiency, performance density and throughput performance are listed for the different solutions. The performance density is only shown for the convolutional (conv) layers.

Technique-MHz	Power efficiency [GOPS/W]	Power E. (conv) [GOPS/W]	Density [GOPS/DSP]	Density [GOPS/kLUT]	Perfor. (conv) [GOPS]	Perfor.(total) [GOPS]	Xilinx device	Fix. [bits]
DNNWEAVER-150	n/a	n/a	0.143	0.59	31.4	n/a	ZC7Z020	16
fpgaConvNet-125	n/a	7.27	0.221	0.91	48.5	12.7	ZC7Z020	16
fpgaConvNet-125	n/a	n/a	0.173	0.71	156	n/a	ZC7Z045	16
Angel-Eye-150	14.2	n/a	0.209	0.86	188	137	ZC7Z045	16
Caffeine-200	10.64	12.4	0.187	1.55	310	266	KU060	16
$CNNA_{16}$-100	6.28	11.86	0.078	0.62	26.4	14.0	ZU3EG	16
$CNNA_{16}$-172	5.99	11.08	0.081	0.52	29.1	15.7	ZU3EG	16
Angel-Eye-214	n/a	24.1	n/a	n/a	85.3	n/a	ZC7Z020	8
$CNNA_8^1$-172	15.22	22.94	0.155	0.66	38.0	25.2	ZU3EG	8
$CNNA_8^2$-172	11.83	22.53	0.110	0.61	39.5	20.7	ZU3EG	8

use any 8-, 16- or 32-bit fixed-point formats when exported from Keras, with the weights autoscaled correctly. A training method was proposed that achieved high levels of inference accuracies, both when using fixed-point and floating-point weights. The VGG16 architecture chosen for testing in this paper was able to perform inference in 2.0 s per image when using the fixed-point format Q2.14 and 1.2 s when using fixed-point format Q2.6. The IP core alone consumed a peak power of 4.1 W with a mean power between 1.5–2.7 W and had a power efficiency between 6.0–15.2 GOPS/W depending on the fixed-point format.

Compared to similar state-of-the-art solutions for mapping the VGG16 network to Xilinx platforms, our solution demonstrates a comparable energy efficiency, especially for convolutional layers. In future work, the CNNA should be extended to support special layers to support deep neural networks, such as ResNet, DenseNet, InceptionNet and GooglLeNet. The special layers with irregular dataflow will be implemented in the SW controlling part of the proposed architecture.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

We would like to thank Freia Martensen for language and proofreading the article.

References

[1] A. Krizhevsky, I. Sutskever, G.E. Hinton, ImageNet classification with deep convolutional neural networks, Commun. ACM (2017) http://dx.doi.org/10.1145/3065386.

[2] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, in: 3rd International Conference on Learning Representations, ICLR 2015 - Conference Track Proceedings, 2015, arXiv:1409.1556.

[3] J. Redmon, S. Divvala, R. Girshick, A. Farhadi, You only look once: Unified, real-time object detection, in: Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition, 2016, http://dx.doi.org/10.1109/CVPR.2016.91, arXiv:1506.02640.

[4] S. Ren, K. He, R. Girshick, J. Sun, Faster R-CNN: Towards real-time object detection with region proposal networks, IEEE Trans. Pattern Anal. Mach. Intell. (2017) http://dx.doi.org/10.1109/TPAMI.2016.2577031, arXiv:1506.01497.

[5] K. He, G. Gkioxari, P. Dollar, R. Girshick, Mask R-CNN, IEEE Trans. Pattern Anal. Mach. Intell. (2018) http://dx.doi.org/10.1109/TPAMI.2018.2844175.

[6] A. Shawahna, S.M. Sait, A. El-Maleh, FPGA-Based accelerators of deep learning networks for learning and classification: A review, IEEE Access (2019) http://dx.doi.org/10.1109/ACCESS.2018.2890150, arXiv:1901.00121.

[7] S. Mittal, J.S. Vetter, A survey of methods for analyzing and improving gpu energy efficiency, ACM Comput. Surv. (2014) http://dx.doi.org/10.1145/2636342, arXiv:1404.4629.

[8] S. Mittal, A survey of FPGA-based accelerators for convolutional neural networks, Neural Comput. Appl. (2018) http://dx.doi.org/10.1007/s00521-018-3761-1.

[9] W. Ding, Z. Huang, Z. Huang, L. Tian, H. Wang, S. Feng, Designing efficient accelerator of depthwise separable convolutional neural network on FPGA, J. Syst. Archit. (2019) http://dx.doi.org/10.1016/j.sysarc.2018.12.008.

[10] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, Y. Bengio, Binarized neural networks, in: Advances in Neural Information Processing Systems, 2016, arXiv:1602.02505.

[11] M. Blott, T.B. Preuber, N.J. Fraser, G. Gambardella, K. O'Brien, Y. Umuroglu, M. Leeser, K. Vissers, FinN-R: An end-to-end deep-learning framework for fast exploration of quantized neural networks, ACM Trans. Reconfig. Technol. Syst. (2018) http://dx.doi.org/10.1145/3242897, arXiv:1809.04570.

[12] E. Nurvitadhi, D. Sheffield, J. Sim, A. Mishra, G. Venkatesh, D. Marr, Accelerating binarized neural networks: Comparison of FPGA, CPU, GPU, and ASIC, in: Proceedings of the 2016 International Conference on Field-Programmable Technology, FPT 2016, 2017, http://dx.doi.org/10.1109/FPT.2016.7929192.

[13] R. Zhao, W. Song, W. Zhang, T. Xing, J.-H. Lin, M. Srivastava, R. Gupta, Z. Zhang, Accelerating binarized convolutional neural networks with software-programmable FPGAs, in: Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, in: FPGA '17, ACM, New York, NY, USA, 2017, pp. 15–24, http://dx.doi.org/10.1145/3020078.3021741, URL http://doi.acm.org/10.1145/3020078.3021741.

[14] R. Nane, V.M. Sima, C. Pilato, J. Choi, B. Fort, A. Canis, Y.T. Chen, H. Hsiao, S. Brown, F. Ferrandi, J. Anderson, K. Bertels, A survey and evaluation of FPGA high-level synthesis tools, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. (2016) http://dx.doi.org/10.1109/TCAD.2015.2513673.

[15] H. Li, X. Fan, L. Jiao, W. Cao, X. Zhou, L. Wang, A high performance FPGA-based accelerator for large-scale convolutional neural networks, in: FPL 2016 - 26th International Conference on Field-Programmable Logic and Applications, 2016, http://dx.doi.org/10.1109/FPL.2016.7577308.

[16] F. Chollet, et al., Keras, 2021, https://keras.io (accessed on 2/9-2021).

[17] L. Stornaiuolo, M. Santambrogio, D. Sciuto, On how to efficiently implement deep learning algorithms on pynq platform, in: Proceedings of IEEE Computer Society Annual Symposium on VLSI, ISVLSI, 2018, http://dx.doi.org/10.1109/ISVLSI.2018.00112.

[18] Accellera Systems Initiative, IEEE Standard for Standard SystemC Language Reference Manual, IEEE Std 1666-2011 (Revision of IEEE Std 1666-2005), 2012.

[19] K. Ovtcharov, O. Ruwase, J.-y. Kim, J. Fowers, K. Strauss, E.S. Chung, Accelerating Deep Convolutional Neural Networks Using Specialized Hardware, Microsoft Research Whitepaper, 2015.

[20] D. Gschwend, ZynqNet: An FPGA-accelerated embedded convolutional neural network. URL https://github.com/dgschwend/zynqnet.

[21] S.I. Venieris, A. Kouris, C.S. Bouganis, Toolflows for mapping convolutional neural networks on FPGAS: A survey and future directions, ACM Comput. Surv. (2018) http://dx.doi.org/10.1145/3186332, arXiv:1803.05900.

[22] S.I. Venieris, C.S. Bouganis, FpgaConvNet: A framework for mapping convolutional neural networks on FPGAs, in: Proceedings - 24th IEEE International Symposium on Field-Programmable Custom Computing Machines, FCCM 2016, 2016, http://dx.doi.org/10.1109/FCCM.2016.22.

[23] S.I. Venieris, C.S. Bouganis, FpgaConvNet: Mapping regular and irregular convolutional neural networks on FPGAs, IEEE Trans. Neural Netw. Learn. Syst. (2019) http://dx.doi.org/10.1109/TNNLS.2018.2844093.

[24] H. Sharma, J. Park, D. Mahajan, E. Amaro, J.K. Kim, C. Shao, A. Mishra, H. Esmaeilzadeh, From high-level deep neural models to FPGAS, in: Proceedings of the Annual International Symposium on Microarchitecture, MICRO, 2016, http://dx.doi.org/10.1109/MICRO.2016.7783720.

[25] K. Guo, L. Sui, J. Qiu, J. Yu, J. Wang, S. Yao, S. Han, Y. Wang, H. Yang, Angel-Eye: A complete design flow for mapping CNN onto embedded FPGA, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. (2018) http://dx.doi.org/10.1109/TCAD.2017.2705069.

[26] Y. Wang, J. Xu, Y. Han, H. Li, X. Li, DeepBurning: Automatic generation of FPGA-based learning accelerators for the neural network family, in: Proceedings - Design Automation Conference, 2016, http://dx.doi.org/10.1145/2897937.2898003.

[27] C. Zhang, Z. Fang, P. Zhou, P. Pan, J. Cong, Caffeine: Towards uniformed representation and acceleration for deep convolutional neural networks, in: IEEE/ACM International Conference on Computer-Aided Design, Digest of Technical Papers, ICCAD, 2016, http://dx.doi.org/10.1145/2966986.2967011.

[28] Y. Umuroglu, N.J. Fraser, G. Gambardella, M. Blott, P. Leong, M. Jahre, K. Vissers, FINN: A framework for fast, scalable binarized neural network inference, in: FPGA 2017 - Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, 2017, http://dx.doi.org/10.1145/3020078.3021744, arXiv:1612.07119.

[29] Y.Y. Huang, W.Y. Wang, Deep residual learning for weakly-supervised relation extraction, in: EMNLP 2017 - Conference on Empirical Methods in Natural Language Processing, Proceedings, 2017, http://dx.doi.org/10.18653/v1/d17-1191, arXiv:1707.08866.

[30] G. Huang, Z. Liu, L. Van Der Maaten, K.Q. Weinberger, Densely connected convolutional networks, in: Proceedings - 30th IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017, 2017, http://dx.doi.org/10.1109/CVPR.2017.243, arXiv:1608.06993.

[31] G. Zeng, Y. He, Z. Yu, X. Yang, R. Yang, L. Zhang, InceptionNet/GoogLeNet - going deeper with convolutions, Cvpr (2016) http://dx.doi.org/10.1002/jctb.4820, arXiv:1409.4842.

[32] J.H. Schougaard, K. Bjerge, Source code: Convolutional neural network accelerator, 2021, https://github.com/jonathan93sh/CNNA (accessed on 2/9-2021).

[33] Xilinx, PYNQ: Python productivity for zynq, 2021, URL http://www.pynq.io/ (accessed on 2/9-2021).

[34] Xilinx, UG 902 - Vivado Design Suite User Guide - High-Level Synthesis, 2019th ed., Xilinx, 2021, (accessed on 2/9-2021).

[35] Accellera Systems Initiative, SystemC Synthesizable Subsets, first ed., 2015.

[36] L.H. Crockett, D. Northcote, C. Ramsay, F.D. Robinson, R.W. Stewart, Exploring Zynq® MPSoC With PYNQ and Machine Learning Applications, Strathclyde Academic Media, 2019.

[37] Xilinx, UG761 - AXI Reference Guide, v13.1 ed., 2011.

[38] B. Xu, R. Huang, M. Li, Revise saturated activation functions, 2016, CoRR abs/1602.05980. arXiv:1602.05980. URL http://arxiv.org/abs/1602.05980.

[39] E. Oberstar, Fixed-Point Representation & Fractional Math Revison 1.2, 2007, http://dx.doi.org/10.13140/RG.2.1.3602.8242.

[40] B.J. Wythoff, Backpropagation neural networks: A tutorial, Chemometr. Intell. Lab. Syst. 18 (2) (1993) 115–155, http://dx.doi.org/10.1016/0169-7439(93)80052-J, URL http://www.sciencedirect.com/science/article/pii/016974399380052J.

[41] H. Park, J.H. Lee, Y. Oh, S. Ha, S. Lee, Training deep neural network in limited precision, 2018, CoRR abs/1810.05486. arXiv:1810.05486. URL http://arxiv.org/abs/1810.05486.

[42] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, D. Kalenichenko, Quantization and training of neural networks for efficient integer-arithmetic-only inference, in: The IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2018.

[43] 96 Boards, Ultra96-v2 developer board. URL https://www.96boards.org/product/ultra96/.

[44] Detect, 2017, (accessed on 1/11-2021) URL https://www.syke.fi/en-US/Research_Development/Research_and_development_projects/Projects/DETECT.

[45] A. Krizhevsky, V. Nair, G. Hinton, CIFAR-10 and CIFAR-100 datasets, 2009.

[46] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A.C. Berg, L. Fei-Fei, ImageNet large scale visual recognition challenge, Int. J. Comput. Vis. (IJCV) 115 (3) (2015) 211–252, http://dx.doi.org/10.1007/s11263-015-0816-y.

[47] Y.Z. Shuchang Zhou, Yuxin Wu, Zekun Ni, Xinyu Zhou, He Wen, Dorefa-Net: Training low bitwidth convolu- tional neural networks with low bitwidth gradients, 2008, http://dx.doi.org/10.1145/1449956.1450053, arXiv:1606.06160v3 [Cs.NE] 2 Feb 2018 DoReFa-Net. arXiv:arXiv:1606.06160v3.

[48] Xilinx, Zynq UltraScale+ MPSoC ZCU104 evaluation kit, 2021, (accessed on 2/9-2021). URL https://www.xilinx.com/products/boards-and-kits/zcu104.html.

[49] J. Li, G. Yan, W. Lu, S. Jiang, S. Gong, J. Wu, X. Li, SmartShuttle: Optimizing off-chip memory accesses for deep learning accelerators, in: Proceedings of the 2018 Design, Automation and Test in Europe Conference and Exhibition, DATE 2018, 2018, http://dx.doi.org/10.23919/DATE.2018.8342033.

Kim Bjerge was born in 1965. He has a M.Sc. in Technical Information Technology from Aarhus University, Denmark in 2013 and a B.Sc. in Electronics engineering from 1989. He has 20 years of experience working with embedded systems and software as a developer, manager and consultant in the industry and the Danish Technological Institute. Kim is today associate professor at Aarhus University, School of Engineering teaching and working in the field of applied signal processing, machine learning, digital system design and HW/SW co-design.

Jonathan Horsted Schougaard was born in 1993. He has a M.Sc. in Computer Engineering from Aarhus University, Denmark 2020 and a B.Sc in Electronics engineering from 2018. He has worked with embedded devices with focus on digital HW design in the field of FPGA's specialised in machine learning and signal processing. Jonathan is today working at Grundfos A/S as an Embedded Software Engineer.

Daniel Ejnar Larsen was born in 1993. He has a M.Sc. in Electrical Engineering from Aarhus University, Denmark in 2020. He has experience in working with a broad variety of embedded systems from micro-controllers over FPGAs to Linux systems. Daniel is currently employed as a Software Development Engineer at Grundfos A/S, working with embedded Linux systems.